

DBDOME

dbanalytics service

Low-Level Architecture & Specification

Database Activity Monitoring · DB Health · Governance, Risk & Compliance

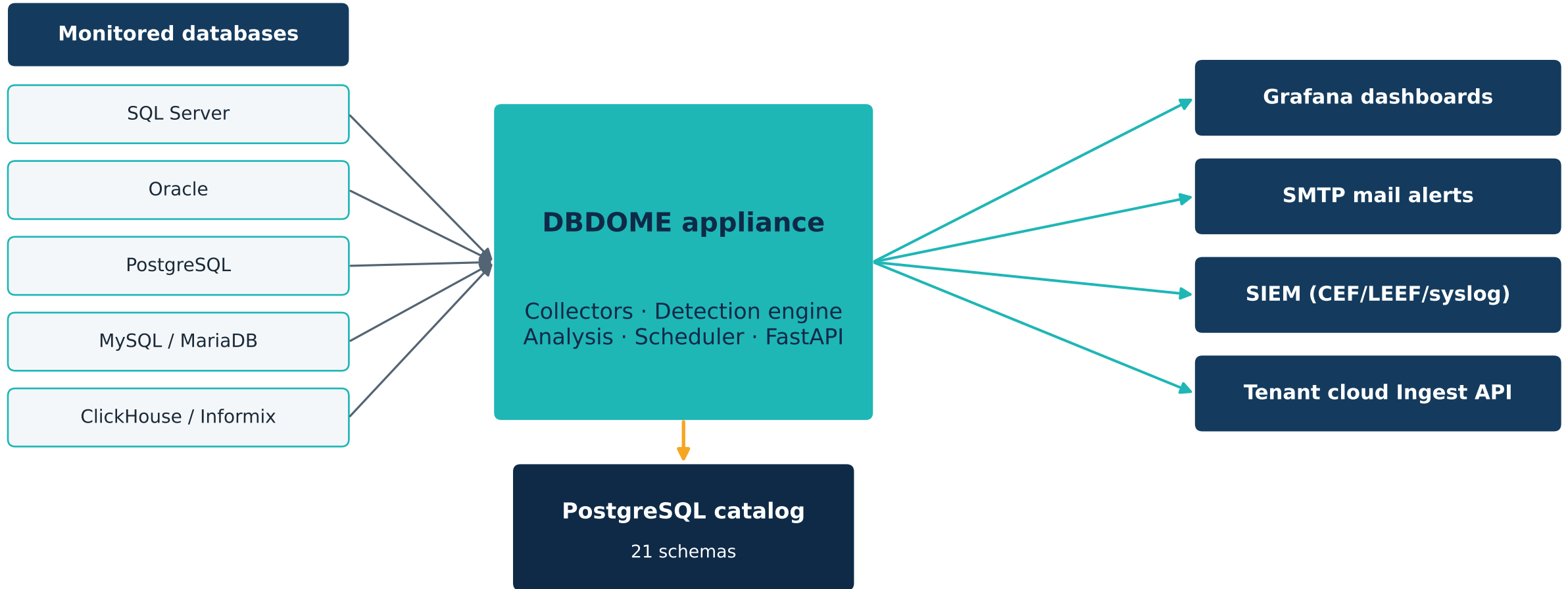
Python appliance (FastAPI + APScheduler) · PostgreSQL catalog · Grafana · Mail / SIEM / Cloud ingest

Multi-vendor: SQL Server · Oracle · PostgreSQL · MySQL · MariaDB · ClickHouse · Informix

What DBDOME is

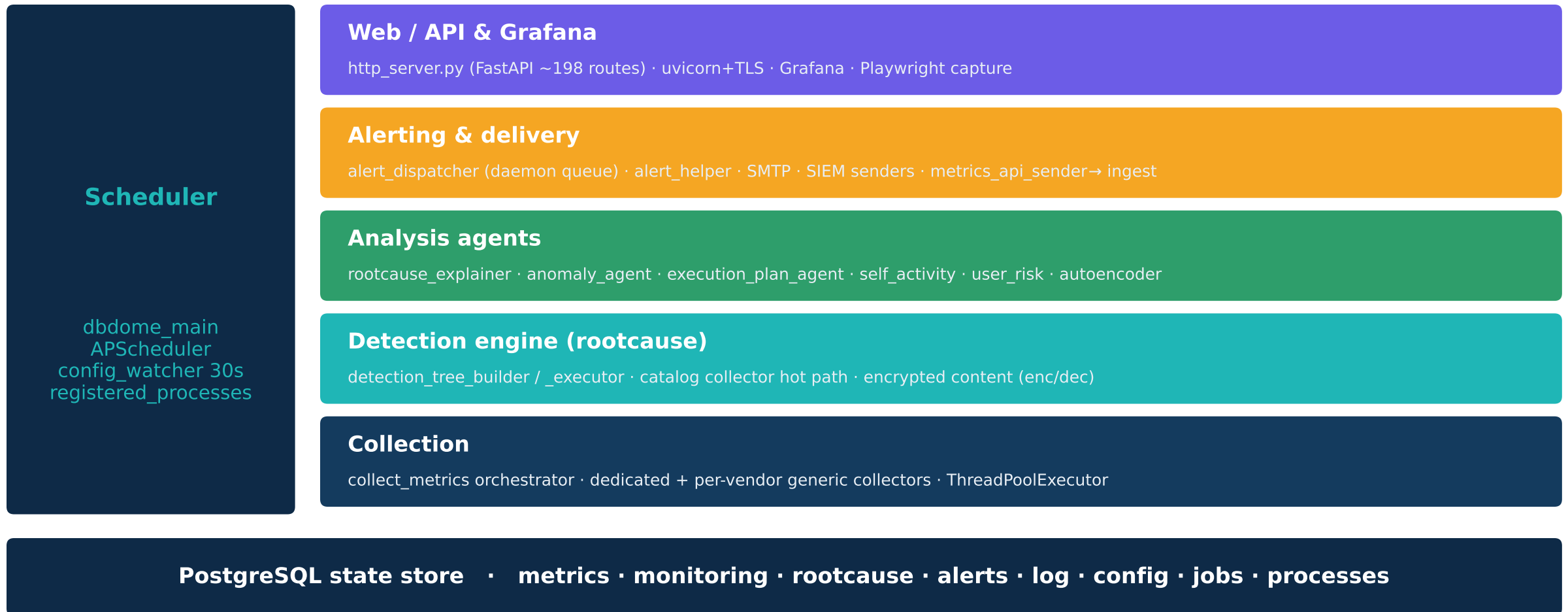
- **Purpose** — an on-prem / appliance platform that continuously monitors production databases for security threats (DAM), health & performance issues, and compliance posture; then alerts, reports, and can auto-remediate.
- **Single process, two faces** — `dbdome_main.py` runs a FastAPI web/API app and an APScheduler background scheduler one process; packaged as a Windows service.
- **PostgreSQL is the brain** — one local PostgreSQL instance (21 schemas) stores registered targets, collected metrics, the encrypted detection catalog, alerts/incidents, audit trail, and all configuration.
- **Detection catalog (rootcause)** — hundreds of per-vendor detections organised as domains → areas → issues → root causes; SQL logic encrypted at rest.
- **Egress** — Grafana dashboards, SMTP mail, SIEM (CEF/LEEF/RFC5424/JSON over UDP/TCP/TLS/HTTP), and an optional multi-tenant cloud ingest API.
- **GRC built in** — masking/tokenization, firewall/blocker, SoD, DDL & privilege audit, retention, tamper-evident hash-chain, and DAM compliance scorecards (PCI-DSS/HIPAA/GDPR/SOC 2/DPDPA/IRDAI/RBI/CERT-In).

System context



Agentless: collectors connect out to targets over native drivers (pyodbc / psycopg2 / oracledb / pymysql); nothing is installed on monitored DB hosts.

Layered architecture



Cross-cutting: utils/ (config_dotenv, secrets_crypto, ssl_cert, alert_resultset, oracle_client, log4dbexpert).

Runtime & process model

- **Entry points** — `dbdome_main.py` starts `uvicorn` (FastAPI `http_server:app`) + `APScheduler BackgroundScheduler` in one process; `dbdome_service.py` wraps it as a Windows service. Flags: `--web-only` / `--scheduler-only` / `--port 8080`.
- **Dynamic jobs** — `config_watcher` runs every 30s → reads `metrics.registered_processes` WHERE `is_active` (name, interval) and adds/removes/reschedules jobs live (no restart). Name→callable via `job_operation_scheduler.get_function_by_name()`.
 - Job policy — `max_instances=1`, `coalesce=True`, `misfire_grace_time=60`.
- **Hard-coded scheduler jobs** — `log_rotation` · `grc_firewall_scan` · `retention & enforcement` · `compliance_reports daily/weekly` · `user_risk_scoring` · `sync_masking_rules` · `threat_response` · `ddl_audit` / `privilege_change` / `sod` / `tls` / `vulnerability scans` · `sql_script_runner` · `dedup_metric_results`.
- **Collection loops** — 6 category entrypoints in `collect_metrics.py`: `operation` · `_sec` · `_perf` · `_hlth` · `_other` · `_critical`.
- **Delivery worker** — `alerts/alert_dispatcher.py`: one daemon thread drains a bounded queue so slow mail/SIEM never blocks the collection hot path. Graceful `SIGTERM/SIGINT` handlers.

Collection layer

- **Orchestrator** — `collection/collect_metrics.py` reads `metrics.v_servers_routines_active` (server, creds, driver, routine, interval, port, oracle_dsn, auth_type, service_name).
- **Dispatch** — `_dispatch_full_row()` matches `routine_name` → collector, run under a `ThreadPoolExecutor` (`DBEXPERT_COLLECTOR_PARALLELISM`, default 8). On success → `jobs.job_handler.job_update_next_run_time + job_history_write`.
- **Two collector families:**
 - Dedicated MSSQL collectors — `collection/mssql/*` (active_sessions, tcp_connection, network_io, database_performance, sql_injection, sensitive_data, privileged_logins, latency, event_session, hardening...).
 - Per-vendor generic (catalog-driven) — `monitoring_metrics_<vendor>_generic_query.py` runs the rootcause detection views for every active root cause.
- **Drivers** — MSSQL → `pyodbc` (ODBC Driver 18/17); PostgreSQL → `psycopg2`; Oracle → `utils/oracle_client`; MySQL/MariaDB → `pymysql`; ClickHouse/Informix → generic modules.
- **Secrets** — monitored-DB passwords stored Fernet-encrypted (`enc:v1:`), decrypted at connect via `utils.secrets_crypto.decrypt_secret`.

Vendor coverage

- **Active SQL vendors** — `sqlserver` · `oracle` · `postgresql` · `mysql` · `mariadb` (`ACTIVE_VENDORS`); plus `clickhouse`, `informix`, `vertica` registered in `rootcause.vendors`.
- `vendor_slug` is the dispatch key — on `metrics.servers.db_vendor`, `detection_steps.vendor_slug` and `detection_paths.vendor_slug`; every root cause carries per-vendor detection SQL.
- **Catalog also lists non-SQL engines** — document (`mongodb/couchbase`), key-value (`redis`), vector (`chroma/milvus/qdrant/weaviate`), cloud PG (`neon/supabase`), self (`dbanalytics`).

Vendor	Representative detection source
SQL Server	<code>sys.dm_exec_requests</code> / <code>sys.dm_exec_sessions</code> / Extended Events
Oracle	<code>v\$session</code> / <code>v\$transaction</code> / <code>dba_*</code> / <code>gv\$*</code>
PostgreSQL	<code>pg_stat_activity</code> / <code>pg_locks</code> / <code>pg_stat_statements</code>
MySQL / MariaDB	<code>information_schema.PROCESSLIST</code> / <code>performance_schema</code>
ClickHouse	<code>system.*</code> tables via HTTP interface

Detection engine — rootcause catalog



- **Step spec** — content `{"sql":...}` · expected `{"condition":"row_count > 0", description, severity}` · parameters (bind thresholds). `path_type = diagnostic | authored`; per-vendor via `vendor_slug`.
- **Path A** — catalog collector (hot path) — generic collectors join `metrics.v_custom_metrics` ↔ `rootcause.v_rootcauses`, run each RC's detection view (`v_sec_sql_*`, `hlth_sql_sys_*`), compare vs expected → store to `monitoring.general_metric_metadata_results` (GMMR).
- **Path B** — detection tree — `detection_tree_builder` (TREE_SQL) + `detection_tree_executor` walk paths step-by-step per vendor (evaluate_condition on row_count), POST executions to the tenant ingest API.
- **Advisory** — `advisories/monitoring_metrics_advisory.py` → `monitoring.build_flowchart(issue_id)` emits per-issue diagnostic flowcharts for the UI.

Detection content encryption

- **What is encrypted** — `detection_steps.content` / `expected` / `parameters` (the proprietary detection SQL + match logic), stored as an armored PGP message (jsonb string scalar).
- **Mechanism (pgcrypto)** — `enc()` = `armor(pgp_sym_encrypt(val, key, aes256))`; `dec()` (STABLE) decrypts or passes plaintext through; `is_encrypted()` checks the PGP header.
- **Auto-encrypt trigger** — `trg_00_encrypt_detection` BEFORE INSERT/UPDATE OF `content`, `expected`, `parameters`; encrypt on write; if the session has no key the row is left as-is (write never blocked). `enc()` is idempotent.
- **Per-session key** — injected by the app via libpq option `options=-c rootcause.k=<DBDOME_SECRET_KEY>` (same host secret as `secrets_crypto`); never stored in catalog, dump, or role settings.
- **Read path** — `v_rootcauses` and `v_issue_decision_tree` call `dec()` so keyed sessions see plaintext; unkeyed sessions get ciphertext/NULL.
- **Threat model** — defeats `pg_dump`, stolen data files, and read-only monitoring/Grafana roles; does not defeat a local admin with the `.env` key. Migrations 7300 / 7310.

Data model — 21 PostgreSQL schemas

Schema	Holds	Schema	Holds
metrics	servers, custom metrics, thresholds, masking logs	web / users	app users, tokens, RBAC, UI audit
monitoring	GMMR results, views, resultset funcs, scorecards	widget	dashboard/report builder metadata
rootcause	encrypted detection/resolution catalog	reports	html_reports, pdf_reports
public	legacy mirror of the catalog	siem / siem_config	SIEM interface config, IPS events
alerts	alert_log (partitioned), incidents, blocks	meta	schema_migrations ledger
log	immutable audit + compliance views	analysis	suppression rules
config	global params, mail/SIEM, masking, GRC policy	flowchart	issue flowchart visual objects
jobs / processes	scheduler jobs, orchestration, orgs	monitoring_history / action	historic results, action types
threats	login/session/object/metric threats (+history)		

Key tables — metrics & monitoring

- **metrics.servers** — registered monitored DBs: server/db/username, password (Fernet enc:v1:), driver, port, db_vendor, db_version, auth_type, service_name, server_id (uuid).
- **thresholds matrix** — thresholds · metrics_thresholds · servers_thresholds · server_metrics_thresholds · routines_thresholds.
- **metrics.sensitive_columns** — catalogued PII/sensitive columns; feeds masking & tokenization (masking_log, masking_tokens, encryption_log).
- **monitoring.general_metric_metadata_results (GMMR)** — central result store; metric_name = root_cause_id, metric_metadata, metric_metadata_vs_expected; RANGE-partitioned by month (gmmr_YYYY_MM + gmmr_default).
- **monitoring result tables** — active_sessions, active_transactions (+locks/long_queries), database_performance, os_memory, tcp_connections, privileged_logins, weak_passwords.
- **monitoring views** — v_dam_compliance_scorecard, v_dam_regulation_score, v_custom_metrics, v_dbanalytics_alert.

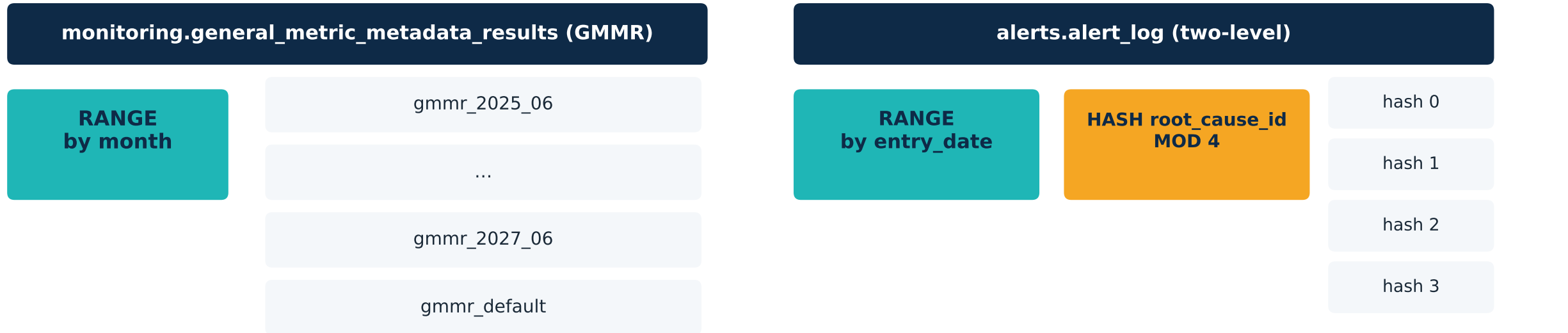
Key tables — rootcause & alerts

- **rootcause catalog** — domains → areas → issues → **root_causes** (codes e.g. PERF-SQL-QE-001-RC01); **detection_paths** (vendor_slug, path_type, is_active, min/max_version, superseded_by_id) → **detection_path_steps** → **detection_steps**.
- **resolution & decision trees** — **resolution_paths/_steps**; **issue_decision_trees/_nodes** (gate_detection_step_id, on_match_rc_id, terminate/drill).
- **decrypting views** — **v_rootcauses**, **v_issue_decision_tree** (call dec()).
- **alerts.alert_log** — raw firing log; TWO-LEVEL partitioned: RANGE(entry_date) monthly → HASH(root_cause_id) MOD 4 (+ hash-subpartitioned DEFAULT). PK (row_id, entry_date, root_cause_id); columns server, risk_level, metadata, metric_query, login_name.
- **alerts.alert_incidents** — durable open→resolved lifecycle (opened_at/last_seen_at, occurrences, resolved_by/type); unique-open index per (server, root_cause_id).
- **alerts.alert_lifecycle_config** — per-severity auto-resolve (critical never; medium 48h; low 24h; default 72h). Also **mail_alert_log**, **blocks/blocker_log**.

Key tables — log (audit) & config

- **log — immutable audit** — operation_log · ddl_audit_log · firewall_audit_log · security_incidents · incidents (+capa/timeline) · cross_border_transfers · attestations · access_review_* · sod_violations · threat_response_log.
- **log — tamper evidence** — hash_chain_checkpoints (hash-chained, tamper-evident audit).
- **log — compliance views** — v_pci_dss_audit · v_hipaa_audit · v_gdpr_audit · v_soc2_audit; report views v_rpt_gdpr/hipaa/pcidss/soc2_*, v_ddl_high_risk, v_blocked_events, v_active_threats.
- **config — runtime configuration** — global_params · thresholds · mail_config / mail_groups · webook_alerts · siem · masking rules & tokens · regulation_profiles · retention policies · firewall policies · sod_rules · threat_response_playbooks · blocked IPs · data_regions · transfer_agreements · risk_register.

Partitioning strategy



- **GMMR** — monthly RANGE partitions with create-ahead + drop-old maintenance → `monitoring.gmmr_maintain(keep_m ahead_months); SECURITY DEFINER (6700); attach fix (7320); gmmr_upsert_latest` trigger.
- **alert_log** — RANGE(entry_date) monthly, each month HASH-subpartitioned MOD 4 on root_cause_id, plus a hash-subpartitioned DEFAULT month → `alerts.alert_log_maintain(keep=12, ahead=3)`. Migration 7340.
- **Why** — bounded scans on hot recent windows, cheap retention by dropping whole partitions, and even write distribution across root causes.

Analysis agents

Agent (analysis/)	What it does
rootcause_explainer.py	Feeds a v_rootcauses row (SQL + condition + resolution) to Claude → DBA/auditor/exec remediation narrative; caches to analysis.rc_explanations.
anomaly_agent.py	z-score anomaly over transaction captures (CPU / duration / after-hours) → maps to a root cause, stores finding + raises alert_log (dry-run unless --commit).
execution_plan_agent.py	For PERF-SQL-TX-010-RC02 captures: parse plan XML (monitoring.parse_execution_plan), rank costliest operators + missing-index recs, write plan_analysis back.
self_activity_agent.py (+filter)	Finds permanent false alarms → proposes analysis.suppression_rules; the deterministic filter enforces them on the hot path.
analyse_user_risk.py	Per-login risk scoring (run_user_risk_scoring).
ai/Autoencoder_v1.py	ML autoencoder model for SQL anomaly detection.

Alerting & incident lifecycle



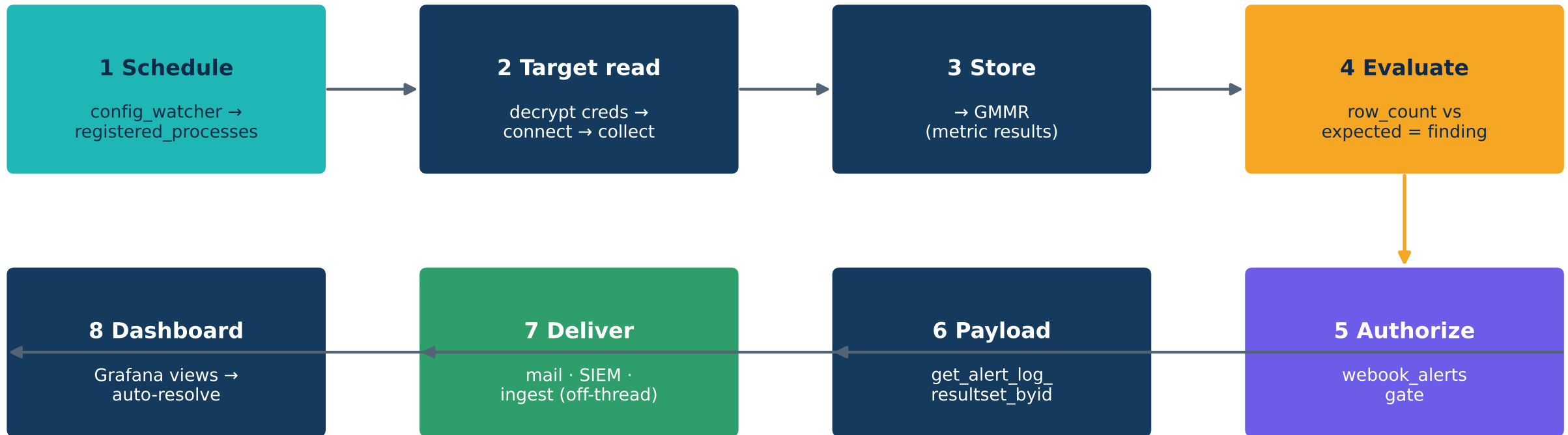
- **Trigger** — a matched detection writes `alerts.alert_log` (server, root_cause_id, risk_level, metadata, login_name); AFTER INSERT trigger opens/refreshes an `alerts.alert_incidents` row.
- **De-duplication** — `alerts.mail_alert_log` enforces "once per hour" per (server, root_cause_id).
- **Authorization** — `config.webook_alerts` (domain_code + risk_level) decides mail/SIEM eligibility; recipients from `config.mail_config` + `mail_groups`.
- **Canonical payload** — `utils/alert_resultset.fetch_alert_resultset` → `monitoring.get_alert_log_resultset_byid(row_id)`: UI, mail and SIEM render identical rows.
- **Closure** — `alerts.resolve_incident` / `resolve_incident_ui` (manual) and `auto_resolve_incidents` per `alert_lifecycle_config`; excluded-login alerts skipped (7290).

Delivery & SIEM integration

- **Off-thread dispatch** — collectors call `alerts.alert_dispatcher.dispatch(fn,...)` (fire-and-forget, bounded queue, one daemon worker).
- **Mail** — `email_utils/smtp_email_sender.py`: alert mail (with/without attachment) and scheduled report delivery.
- **Generic SIEM** — `siem/generic/syslog_sender.siem_send_all` — formats × transports below; config via `config.siem`
`service_name = "<format>:<transport>"`.

Formats	Transports	Vendor defaults	Dedicated senders
CEF · LEEF RFC5424 · RFC3164 JSON	UDP · TCP TLS (mutual) HTTP	splunk → cef/tcp (HEC http) qradar → leef/tcp sentinel/arcsight → cef/tcp elastic → json/tcp	rapid · crowdstrike · wazuh fortianalyzer (inbound IPS) metrics_api_sender → ingest

End-to-end data flow



Components: collect_metrics · secrets_crypto · monitoring.general_metric_metadata_results · alert_log / alert_incidents · config.webook_alerts · utils/alert_resultset · alert_dispatcher → smtp/siem/metrics_api_sender · Grafana + alert_auto_resolve.

Web / HTTP API & Grafana

■ FastAPI — `http_server.py` — ~198 routes served by uvicorn with TLS (`utils/ssl_cert`).

- Config UI/forms — `/serverform` · `/processform` · `/siem_configuration` · `/custom_metrics` · `/mailconfiguration` · `/alert_rules` · `/alert_thresholds` · `/global_params` · `/exclude_logins` · `/app_login_guard`.
- Print/report ops — `/api/print-panel` · `/api/email-panel` · `/capture_report` · `/report_schedule (+toggle)` · `/dam_report` · `/api/print-dam`.
- Ops endpoints — `POST /api/mailconfiguration/test` · `/api/ssl/status|upload|revert` · `/api/alerts/incidents|open|log/{id}` · `/api/resolve-incident` · `/api/ips/*` · `/api/retention/*` · `/api/blocker/*`.
- Data protection & GRC — `/dp_* mask/anonymize/tokenize` · `/mask_column` · `/encrypt_column` · `/grc/policies|audit-log|compliance-reports|risk-dashboard`.

■ Grafana — runs as a separate service (**DBDOME_Grafana**, watched by `internal_health_monitor`); dashboards are data-driven off the monitoring/alerts views.

- Authoring & capture — panel JSON from `widgets/monitoring_dashboard_widget_export.py` + pattern builders; PDF/PNG capture via Playwright (`print_utils/dashboard_capture.py`).

Security & compliance

- **Secret encryption** — `utils/secrets_crypto.py`: Fernet (AES-128-CBC + HMAC), values prefixed `enc:v1:`; key `DBDOME_SECRET_KEY` in host `.env` (outside DB). Protects `metrics.servers.password` + SMTP password.
- **Detection-catalog encryption** — `pgcrypto`-encrypted rootcause content, per-session key (see encryption slide).
- **TLS & code signing** — `certification/certification.py` generates self-signed RSA-2048/SHA-256 (CN=dbdome), exports `.pfx`; customer cert upload via `/api/ssl`; `sign_dbdome.ps1`.
- **RBAC (6690)** — `dbdome_engine` = sole principal with rootcause access (owns catalog + views, `pg_monitor`); `dbdome_grafana_ro` = read-only, customer schemas only, NO rootcause; transitional mon roles revoked at cutover.
- **Data protection** — column tokenization (SHA2_256, [unencrypted] backup, dry-run switch); dynamic/static masking rules & tokens.
- **DAM compliance** — `v_dam_compliance_scorecard` (0-100 per control, banded) + `v_dam_regulation_score`; templates D
· DPDPA · IRDAI · RBI · CERT-In; mapped to PCI-DSS/HIPAA/SOX/GDPR/SOC 2; tamper-evident hash-chain audit.

Build & deployment

- **Package — PyInstaller onedir (DBDOME_dbanalytics.spec); two exes: dbdome_dbanalytics (console) + dbdome_service (Windows service); entry dbdome_main.py.**
 - Build requirement — MUST be built with the C:\apps\venv314 interpreter (correct deps incl. reportlab); force-bundles pywin32 DLLs, pycpg2 native libs, pytz; UPX off; torch/transformers excluded.
- **Install targets — deploy to C:\ProgramData\DBDOME\bin (preserve bin\.env); packaged for C:\installs\dbdome_setup (full) and C:\installs\dbdome_update (incremental).**
 - Deploy scripts — deploy_redeploy.ps1 / deploy_all_three.ps1; code signing via certification/sign_dbdome.ps1.
- **Schema at runtime — sql_scripts/ (base_install/ + numbered idempotent migrations, tracked in meta.schema_migrations) applied by the sql_script_runner job on startup + daily.**
- **.env keys — PG_HOST/PORT/USER/PASSWORD/DB · DBDOME_SECRET_KEY · ORG_IP · WEB_SSL_ENABLED · ORACLE_CLIENT · GRAFANA_EXE · (config.global_params INGEST_URL/INGEST_API_KEY for cloud push).**

Spec appendix & references

■ Notable functions:

- `monitoring.get_alert_log_resultset_byid(row_id)` / `get_root_cause_resultset(rc)` — canonical alert & RC result rows.
- `monitoring.gmmr_maintain()` / `alerts.alert_log_maintain()` — partition create-ahead + retention.
- `monitoring.parse_execution_plan(text|row_id)` — SQL Server showplan XML → operator rows.
- `alerts.resolve_incident` / `resolve_incident_ui` / `auto_resolve_incidents` — incident lifecycle API.

■ Source docs (repo) — `docs/DBDOME_dbanalytics_service_SPEC.md` (21 schemas, entry points, build) · `docs/DBDOME_FUNCTIONALITIES.md` (198 routes, 35 engines) · `docs/DBDOME_PANEL_QUERIES.md` (88 dashboards, 60 queries).

■ Key code map — `dbdome_main.py` · `collection/collect_metrics.py` · `processes/detection_tree_{builder,executor}.py` · `alerts/{alert_dispatcher,alert_helper,metrics_api_sender}.py` · `utils/alert_resultset.py` · `siem/generic/syslog_sender.py` · `analysis/*.py` · `http_server.py` · `sql_scripts/`.

■ Migrations of note — 7300/7310 content encryption · 7340 alert_log partitioning · 7280 DAM scorecard · 6690 RBAC hardening · 6550 incident lifecycle.